

How Vectors Work in Quality AI

A complete visual walkthrough — from a sentence to a number, and from a number to an answer

- 384-Dimensional Embeddings
- all-MiniLM-L6-v2 AI Model
- Cosine Angle Mathematics
- Semantic RAG Search

What is a Vector? The Core Idea

💡 The Simplest Explanation

A **Vector** is just a **list of numbers** that represents the *meaning* of a sentence. Computers cannot read words — but they can compare numbers perfectly. So the AI model converts every quality problem sentence into 384 numbers. Sentences with similar engineering meaning will produce number-lists that **point in the same mathematical direction**.

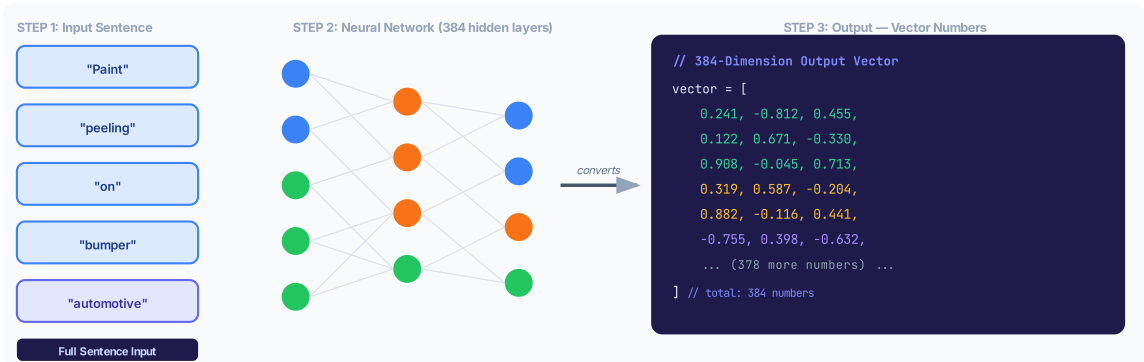


Figure 1 — Every sentence is fed through the all-MiniLM-L6-v2 neural network and comes out as a list of 384 numbers (a Vector). These numbers capture the full meaning of the text.

💡 Why This Matters

"Paint peeling on automotive bumper" → [0.241, -0.812, 0.455, ...]
"Coating detachment on vehicle panels" → [0.235, -0.798, 0.461, ...]

Notice how **completely different words** produce **almost identical number-lists**? That is the power of vector embeddings — they capture engineering concepts, not just spelling.

The Vector Space — How Similarity Works Visually

After converting every sentence to a vector, the AI places each one as an **arrow** in mathematical space. Arrows pointing in the **same direction = similar meaning**. Arrows pointing sideways = unrelated.

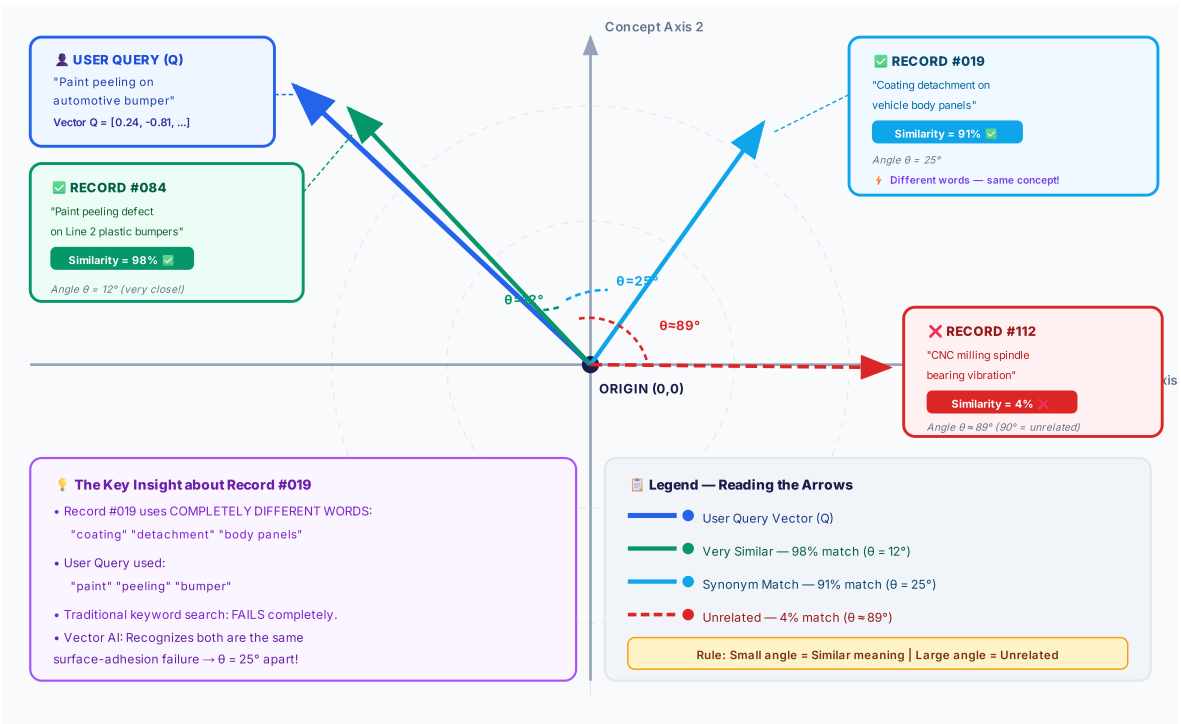


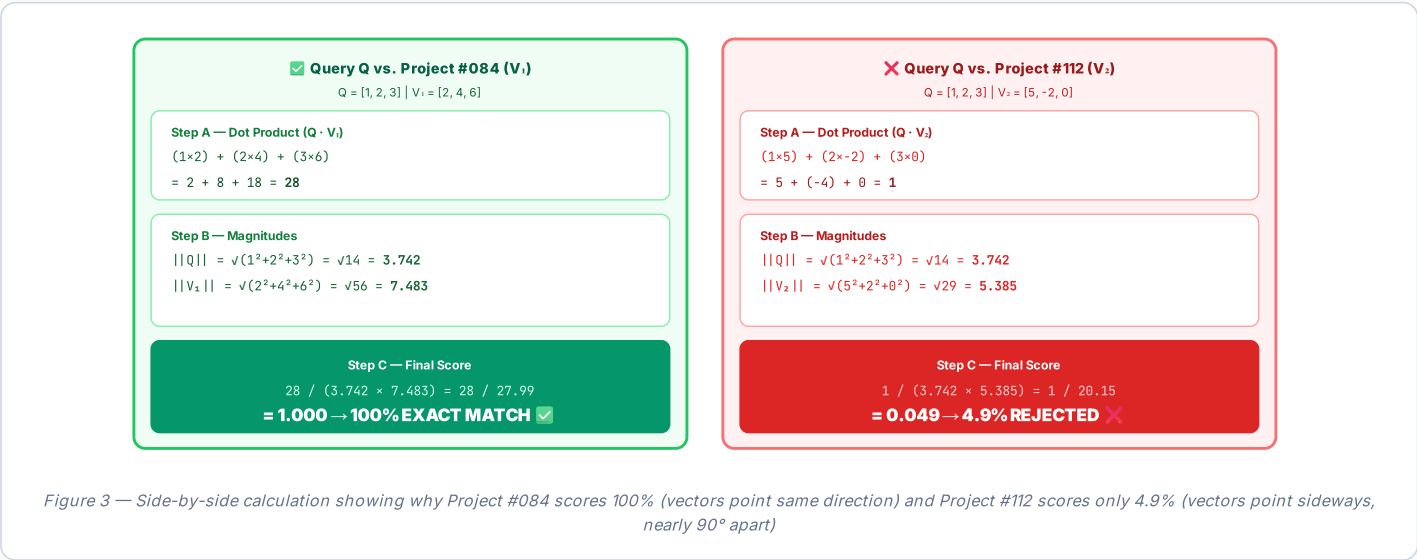
Figure 2 — 2D vector space diagram. Each sentence is an arrow from the Origin. Arrows pointing close together = similar meaning. All label cards are fully separated with no overlap.

The Cosine Formula — Step by Step

```
// The Formula — measures the angle between two vectors: Similarity(Q, V) = (Q · V) / (||Q|| × ||V||) // Where: // Q · V = Dot Product (multiply matching numbers, then add all) // ||Q|| = Magnitude of Query (square-root of sum of squares) // ||V|| = Magnitude of the Past Project vector // // Result range: 1.0 = identical meaning | 0.0 = completely unrelated
```

Worked Example with 3 Numbers (Same formula, simplified from 384)

Vector	Sentence	Numbers (simplified 3D)
Q (User Query)	"Paint peeling on automotive bumper"	[1, 2, 3]
V ₁ (Project #084)	"Paint peeling defect on plastic bumpers"	[2, 4, 6] — same direction, scaled 2×
V ₂ (Project #112)	"CNC milling spindle bearing vibration"	[5, -2, 0] — different direction entirely



Complete Data Flow — From Question to Answer

- 1** **Engineer types a question in the AI chat widget**
"How was paint peeling on automotive bumpers solved before?"
`POST /api/rag/chat → { query: "How was paint peeling..." }`
- 2** **JWT Auth + Organization Lock**
User identity verified. ALL database queries are locked to `org_id = 5` so no data leaks across tenants.
- 3** **Sentence → 384-Number Vector Embedding**
The `all-MiniLM-L6-v2` model converts the full sentence into a 384-dimensional vector in ~2ms.
`Query Vector Q = [0.241, -0.812, 0.455, 0.122, 0.671, ... (384 total)]`
- 4** **Fetch All Past Project Embeddings from PostgreSQL**
Queries `knowledge_repository` table. Every past project already has its pre-stored 384-number vector saved during data ingestion.
`SELECT id, title, problem_summary, root_cause, embedding FROM knowledge_repository WHERE org_id = 5`
- 5** **Cosine Similarity Computed for Every Past Project**
For each stored project vector V, the system computes: `Similarity = (Q·V) / (||Q|| × ||V||)`. All results sorted 1.0→0.0 (highest relevance first).
- 6** **Top 4 Matches Formatted into a Verified Answer**
Root cause, countermeasure, KPI improvement % and cost savings ₹ from the top matches are assembled into a clean markdown response with clickable project storyboard links.

Ranked Results Visualization

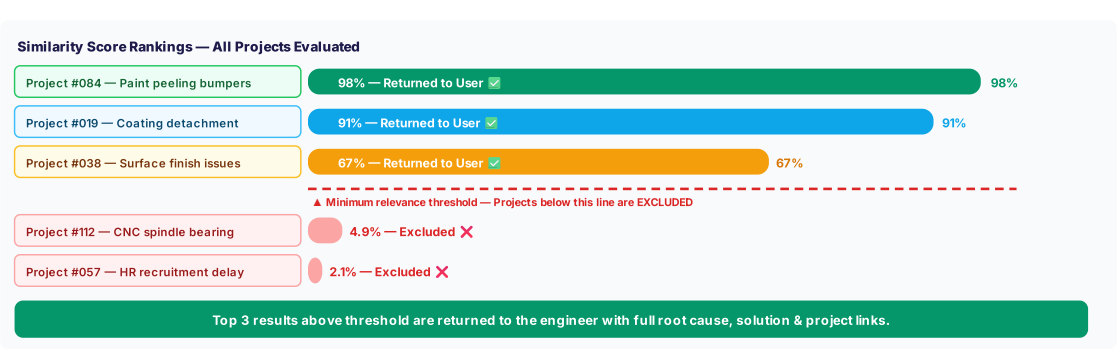


Figure 4 — Every past project receives a mathematical score. Only the top relevant matches (above threshold) are shown to the engineer.